

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

Practical Lab 06

Modbus Communication and Industrial Protocol Analysis

Item	Details
Course	ACICP301 – ICS & SCADA Systems (Practical Laboratories)
Lab	Practical Lab 06: Modbus Communication and Industrial Protocol Analysis
Platform	ICDFA Offline ICS/SCADA Virtual Laboratory – OpenPLC, FUXA & Wireshark
Due	02 Oct 2026, 11:59 PM WAT
Practical Pass	50%
Student Name	Nathaniel Temitope Ogunjobi
Registration Number	C11/26/ACIS/17372
Date	01 September 2026

Authorization & Safety: this practical was performed only within the ICDFA-provided PLC, SCADA and Kali virtual machines, on the instructor-specified isolated/host-only network, with a clean starting snapshot preserved before beginning. Wireshark was used only to capture Modbus traffic generated within this authorised laboratory topology; no production, bridged or external network was scanned or captured.

System Description

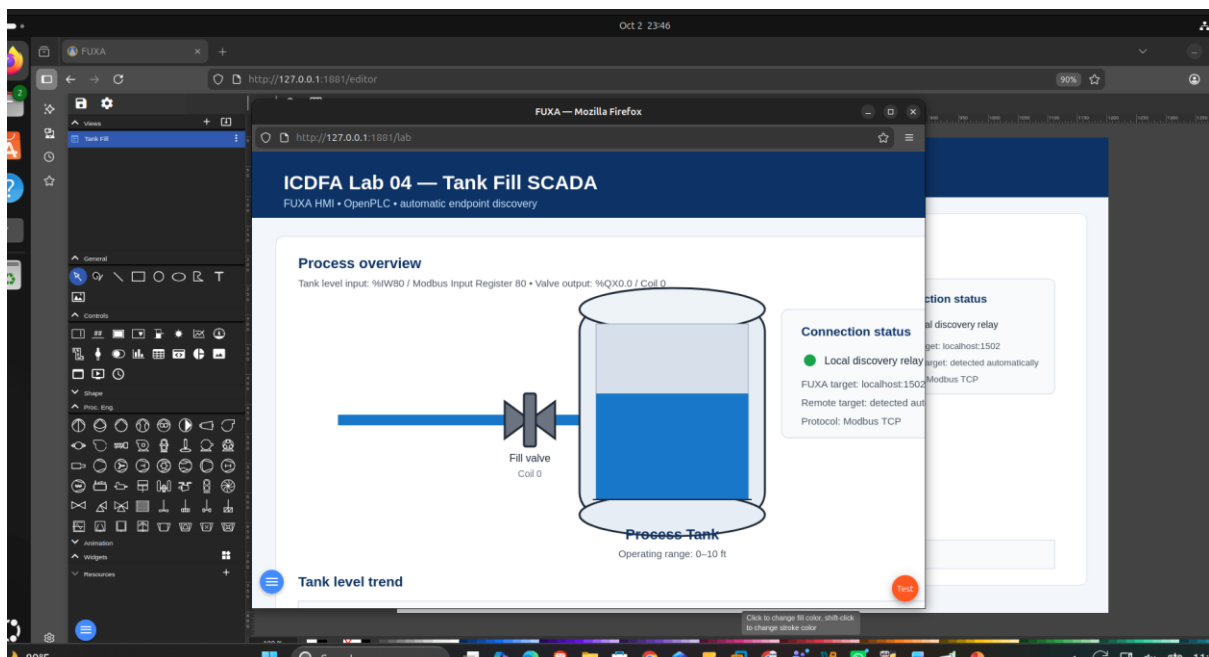
This report documents the capture and analysis of Modbus TCP communication between the FUXA SCADA/HMI system and OpenPLC Runtime for the automated tank-fill process built in the preceding laboratories. Three virtual machines were used: the PLC VM running OpenPLC Runtime and the Tank Fill Sim program, the SCADA VM running the FUXA HMI as the Modbus client, and an analyst VM (Kali) running Wireshark on the SCADA-side network segment to observe the resulting traffic.

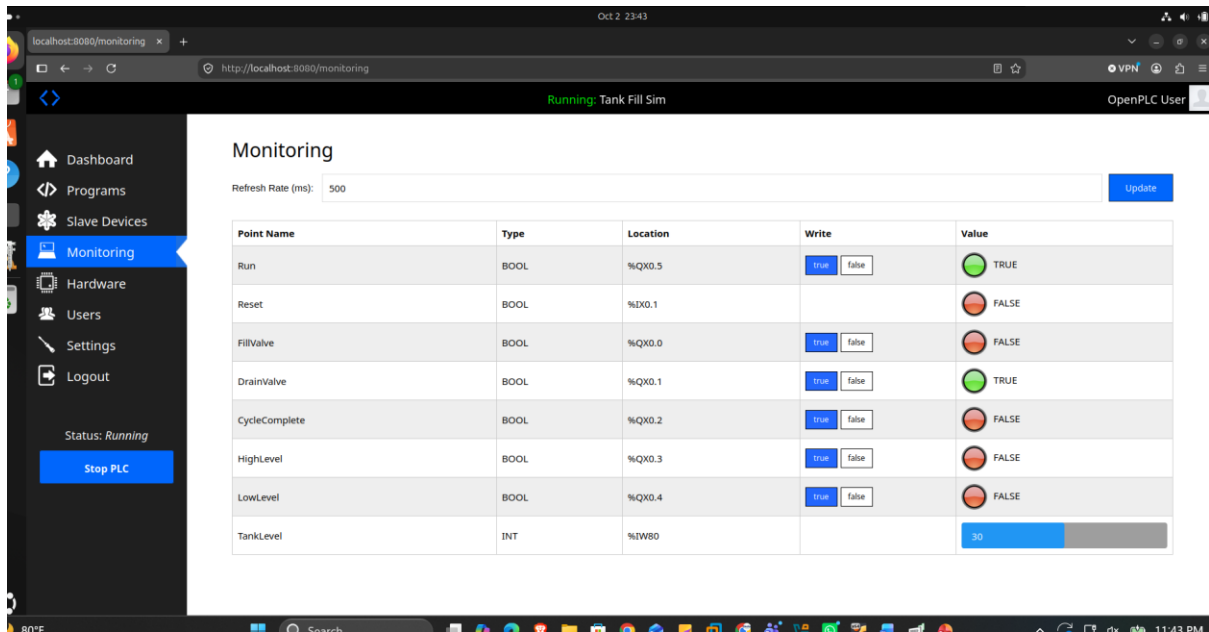
System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password
SCADA VM	192.168.2.10	scada	password
Analyst / Kali VM	Instructor-provided	kali	kali

In this deployment, FUXA is the Modbus client: it polls OpenPLC Runtime, the Modbus server, for the Tank Level and Tank Valve Open values on a regular scan interval. The objective of this practical was to observe that polling directly at the packet level, identify the function codes and registers involved, and relate at least one captured value back to the running process.

Evidence A: OpenPLC and FUXA Process Operation

On the PLC VM, OpenPLC Runtime was opened at <http://localhost:8080/>, signed into, and the TankFillSim.st program (from the Tank Fill Sim directory) was uploaded, named Tank Fill Sim, and started from the Dashboard, with status confirmed as Running. On the SCADA VM, FUXA was started and the previously built TankFillTest.json HMI project was opened at <http://127.0.0.1:1881/editor>. The HMI was run and the tank level was observed changing between its fill and drain limits, confirming the simulation was active before any traffic capture began.





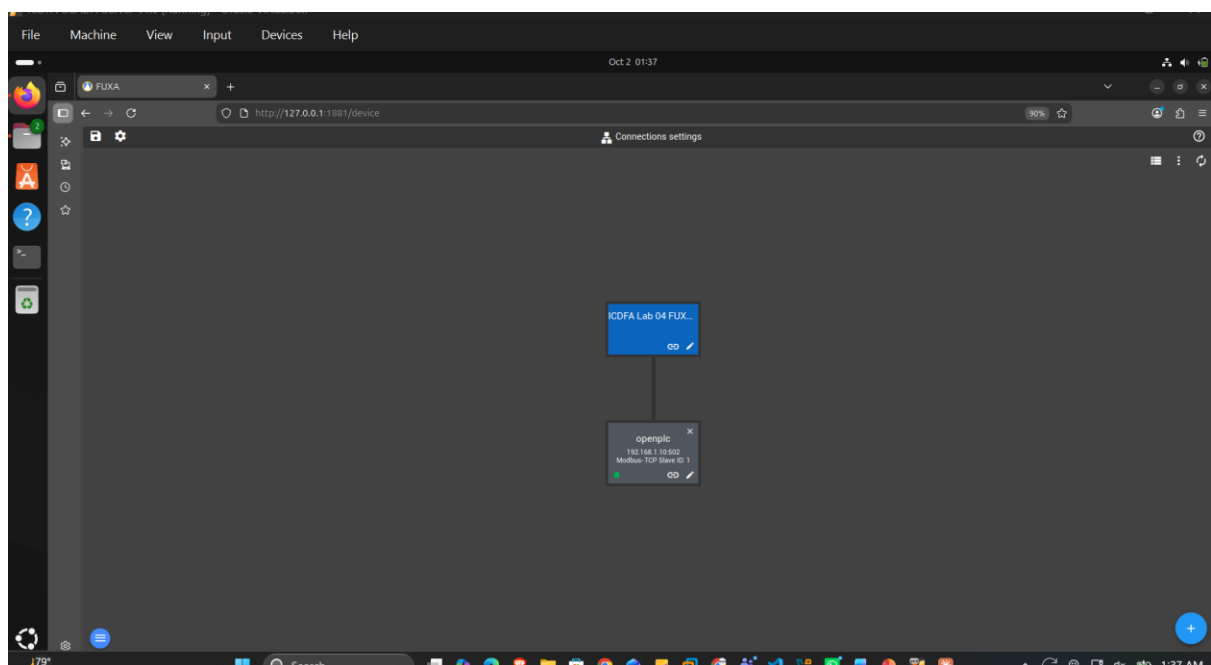
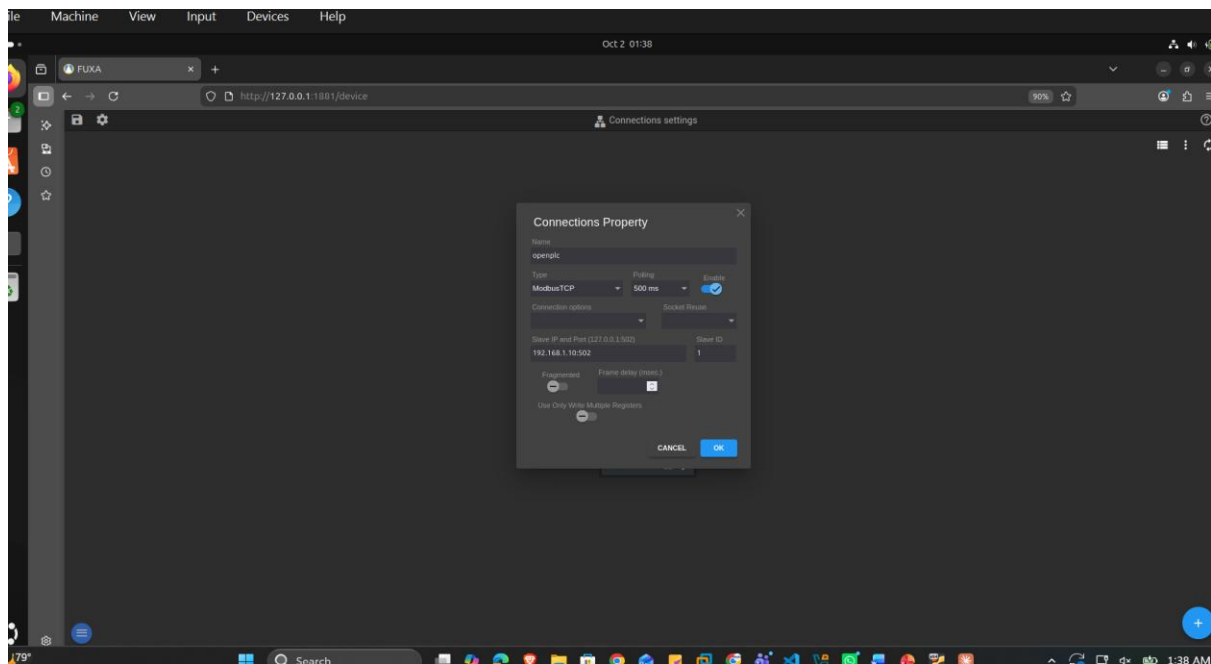
a screenshot of the FUXA HMI running against OpenPLC, showing the tank actively filling or draining

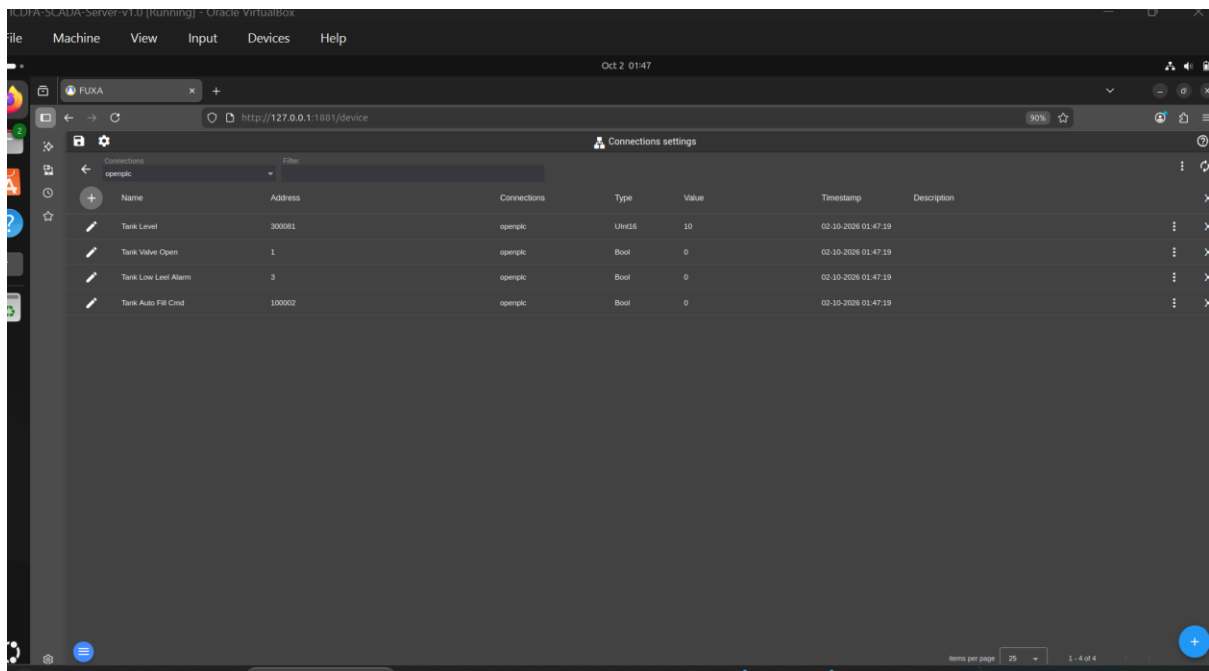
Evidence B: Modbus TCP Connection and OpenPLC Address Identification

In FUXA, under Edit Project, Setup, Connections, the openplc device's Slave IP and Port was confirmed as 192.168.1.10:502, with the green connection indicator confirming an active link to the PLC. On the PLC VM, OpenPLC Runtime's Monitoring tab was used to identify the internal addresses behind the two HMI-facing tags: %QX0.0 for Tank Valve Open (a discrete output coil) and %IW80 for Tank Level (an analog input register).

Modbus Data Type	Usage	PLC Address	Modbus Register	Range
Discrete Output Coils	Digital Outputs	%QX0.0 – %QX99.7	0–799	0 or 1
Discrete Input Contacts	Digital Inputs	%IX0.0 – %IX99.7	0–799	0 or 1
Analog Input Registers	Analog Inputs	%IW0 – %IW1023	0–1023	0–65535
Holding Registers	Analog Outputs	%QW0 – %QW1023	0–1023	0–65535
Holding Registers	Memory (16-bit)	%MW0 – %MW1023	1024–2048	0–65535

Tank Valve Open sits in the discrete output coil range at address 0 (%QX0.0), read and written with the single-bit coil function codes. Tank Level sits in the analog input register range at address 80 (%IW80), read with the 16-bit input-register function code, which is why it appears in the capture as a Function Code 4 exchange on register 80.

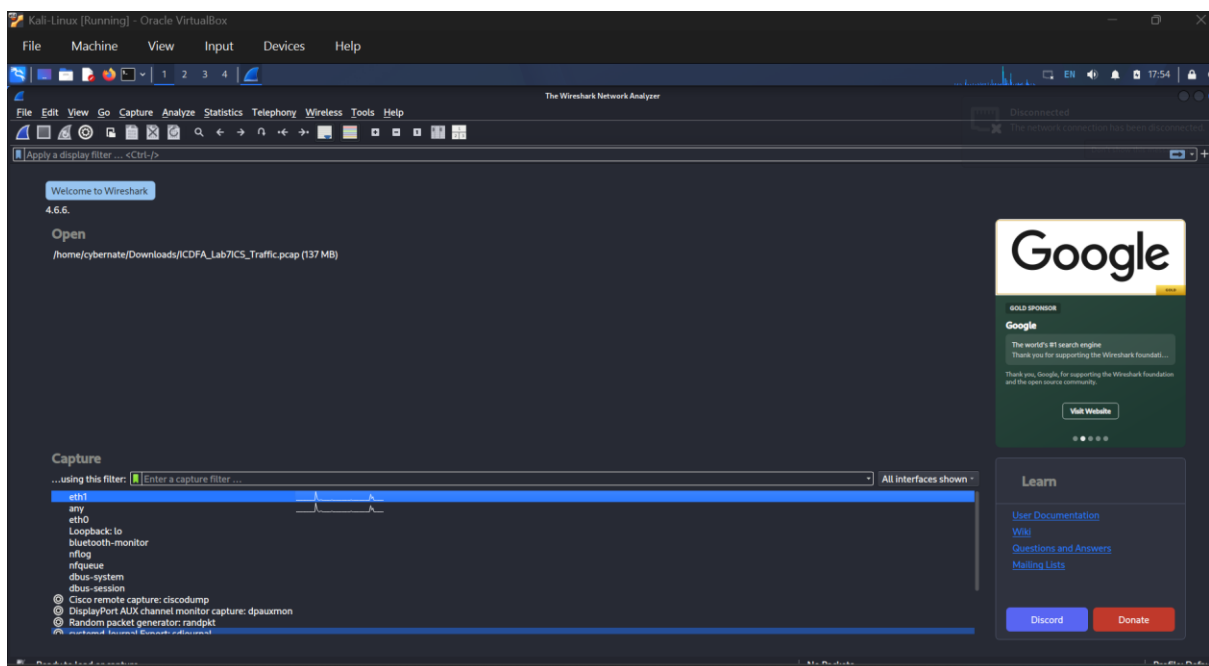


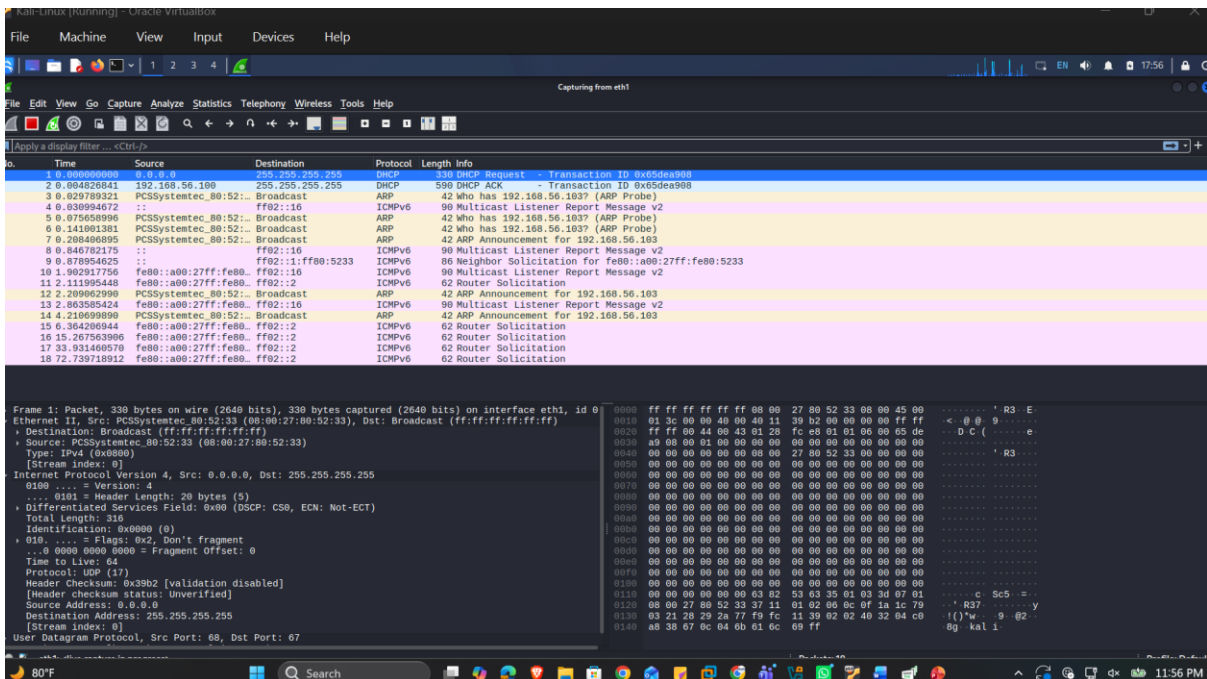
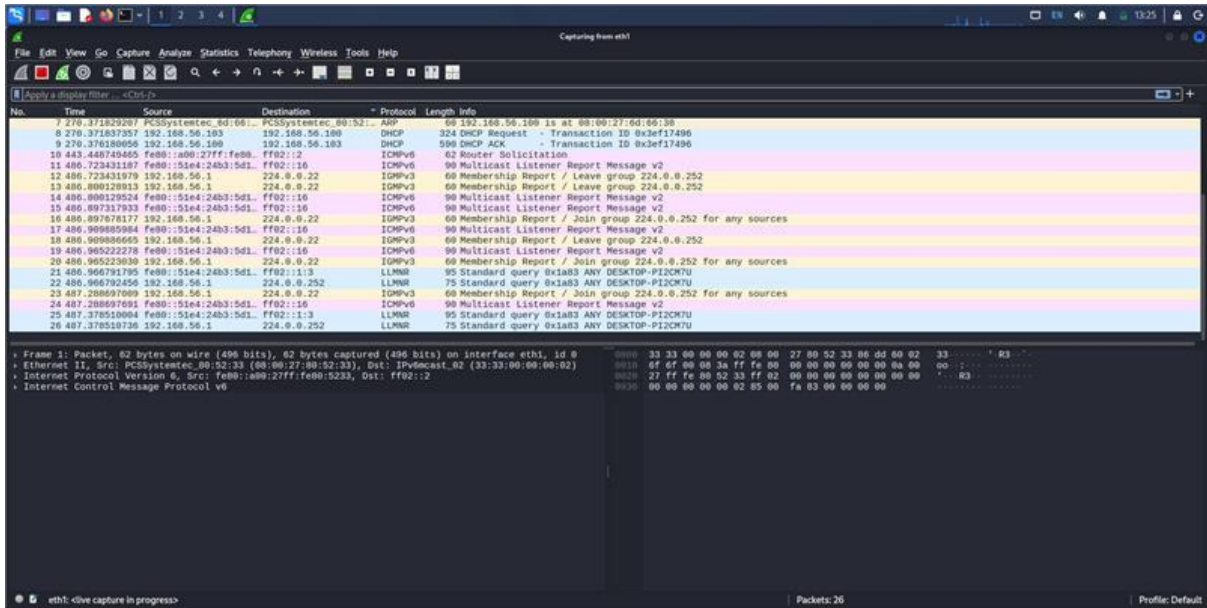


screenshot of the FUXA device connection settings (Slave IP/Port, green status) and, if possible, the OpenPLC Monitoring tab showing the %QX0.0 / %IW80 addresses

Evidence C: Wireshark Interface Selection and Modbus Filter

On the Kali VM, Wireshark was opened and the eth1 interface, carrying the 192.168.2.0/24 SCADA-side segment, was selected so only PLC/SCADA traffic on the lab topology was visible. Before capturing, both other wired network interfaces on the Kali VM were disconnected to avoid picking up unrelated traffic. The display filter modbus was applied (refined with tcp.port == 502 where needed) so the packet list showed only Modbus TCP exchanges.





Evidence F: Interpretation of a Captured Process Value

A Function Code 4 (Read Input Registers) response referencing Register 80 was located in the capture. Register 80 corresponds to %IW80, the Tank Level input register identified in Evidence B. In the reference walkthrough for this teaching image, this exchange carried a value of 6, corresponding to a live tank level of 6 ft at the moment of capture; the precise value in your own capture will depend on where in the fill/drain cycle the process was when the packet was taken.

This single response demonstrates the full path of one industrial data point: the level transmitter's reading is scanned into OpenPLC's %IW80 memory location, FUXA's Modbus client polls that location with a Function Code 4 request against register 80, and OpenPLC's Modbus server responds with the current register value, which FUXA then renders on the HMI's tank-level bar gauge.

Evidence G: Modbus Function Code Reference and TCP vs RTU Comparison

Code	Function	Purpose
01	Read Coils	Read the status of one or more discrete output coils.
02	Read Discrete Inputs	Read the status of one or more discrete input contacts.
03	Read Holding Registers	Read one or more 16-bit holding (analog output/memory) registers.
04	Read Input Registers	Read one or more 16-bit input registers, such as Tank Level at register 80.
05	Write Single Coil	Change the state of a single discrete output coil, such as Tank Valve Open.
06	Write Single Holding Register	Write a new value to a single 16-bit holding register.
15	Write Multiple Coils	Change the state of a contiguous block of discrete output coils in one request.
16	Write Multiple Holding Registers	Write new values to a contiguous block of holding registers in one request.

Modbus TCP versus Modbus RTU

Modbus TCP and Modbus RTU are both implementations of the same Modbus data-organisation protocol, differing in how that data is transported. Modbus RTU is a serial protocol, running over RS232 (single node) or RS485 (multi-node) physical connections, using a Primary/Secondary (Master/Slave) polling architecture where each device on the shared serial bus has a unique address; it is common on older or simpler industrial devices where transmission speed is not critical. Modbus TCP runs over Ethernet using standard IP networking, using a Client/Server model where each device has its own IP address, as used throughout this lab (FUXA at 192.168.2.10 as the client, OpenPLC at 192.168.1.10 as the server on port 502); it is used in modern industrial applications where higher-speed data transmission matters. The function codes, register types and addressing scheme are shared between both transports; only the physical and transport layers differ.

Evidence H: Saved Packet Capture File

The Wireshark capture was saved as a .pcapng file and is submitted alongside this report, named to match this report's registration-number convention C11-26-ACIS-17372_LAB06_capture.pcapng. The saved file contains only traffic generated within the authorized lab topology during the periods described in Evidence D.

Knowledge Check

1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Modbus RTU transmits data serially over RS232 or RS485 physical connections, with devices sharing a bus and identified by a unique serial address in a Primary/Secondary polling scheme. Modbus TCP instead runs over standard Ethernet networking, with each device addressed by its own IP address and communicating over TCP port 502 in a Client/Server model. The underlying Modbus data structure, function codes and registers are the same; only the physical and transport mechanism differs.

2. Why does Modbus use function codes?

A function code tells the server which category of data the client wants to act on, such as a coil, discrete input, holding register or input register, and what action to take on it, such as a read or a write. Because a single PLC exposes many different data areas at overlapping-looking addresses, the function code is what removes the ambiguity, letting the same protocol frame carry precise instructions for many different kinds of process data over one connection.

3. What does Function Code 4 request/read?

Function Code 4 requests a Read Input Registers operation, returning the 16-bit value of one or more analog input registers. In this lab it is the function code used by FUXA to read register 80 (%IW80), the Tank Level value, from OpenPLC.

4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Standard Modbus TCP includes no built-in authentication or encryption, so anyone able to observe traffic on the network segment, exactly as Wireshark did in this lab, can read live process values without needing any credentials. More seriously, the same lack of protection means an attacker positioned on that network could craft and send their own Modbus write requests, such as a Write Single Coil against Tank Valve Open, and have the PLC act on them as if they came from the legitimate SCADA client. This is why unprotected industrial protocol traffic is treated as a genuine operational risk rather than only a confidentiality concern, and why ICS networks are segmented and monitored rather than left open the way this isolated lab topology was for teaching purposes.

Student Declaration

- I confirm that this submission is my own work and that all sources, screenshots, simulations, and calculations used have been honestly presented.
- I understand that unsafe electrical practice, copied work, or fabricated screenshots may attract disciplinary action.

Student Name	Nathaniel Temitope Ogunjobi
Registration Number	C11/26/ACIS/17372
Signature	
Date	02 October 2026